

Solving The Heat equation Across Irregular Domains

A project report

submitted by

Jacob Martin & Ben Pentecost

(2024MTH555)

under the supervision of

Dr. Chuan Li



Department of Mathematics

West Chester University

May 2024

Contents

0.1	Introduction	2
0.2	Research Approach	5
0.2.1	Order of Operation	5
0.2.2	Glossary	6
0.3	Application Prototype	7
0.3.1	Description	7
0.3.2	Usage	8
0.4	Interface Reconstruction	11
0.4.1	Natural Cubic Spline Interpolation	11
0.4.2	Results	13
0.5	Five Numerical Treatments	15
0.5.1	SetS	16
0.5.2	GammaX	18
0.5.3	SeekVP	20
0.5.4	GetNormal	22
0.6	Method	24
0.6.1	Mathematical Models	24
0.6.2	Model Applications	25
0.6.3	AMIB Method	26
0.6.4	Interface Points, Fictitious Points, and Vertical Points	26
0.6.5	Fictitious Value Representations at Fictitious Points	28
0.6.6	The Augmented System	28
0.7	Results	30
0.8	Conclusion	34

0.1 Introduction

Cutting Edge Cancer Research

Magnetic fluid hyperthermia (MFH) is a medical technique that uses magnetic nanoparticles to generate heat in a targeted area of the body for therapeutic purposes. These nanoparticles are typically made of materials like iron oxide and are designed to respond to an alternating magnetic field. When exposed to this field, the nanoparticles rapidly change their magnetic orientation, which creates heat and destroys the surrounding tissue.

In MFH, the magnetic nanoparticles are first administered to the patient, often through injection into the bloodstream or directly into the tumor site. Once the nanoparticles accumulate in the target area, an alternating magnetic field is applied externally using a device called a magnetic field generator. This field causes the nanoparticles to heat up, selectively raising the temperature of the target tissue while minimizing damage to surrounding healthy tissue.

MFH is being studied for its potential in treating various medical conditions, with a primary focus on cancer therapy. The localized heating induced by MFH can help destroy cancer cells directly through hyperthermia (heat-induced cell death) and can also enhance the effectiveness of other cancer treatments like chemotherapy and radiation therapy. MFH is also being explored for its potential in treating other conditions, such as infections and inflammation.

Studies on MFH are ongoing, with researchers investigating its effectiveness, safety, and optimal application methods. Some studies have shown promising results, particularly in combination with other therapies, but further research is needed to fully understand the potential benefits and limitations of MFH across different medical applications.

Mathematical Approach to Aid MFH

There are still many challenges to overcome until MFH is ready for wide-scale use by physicians. Some of the few key challenges are the following: temperature control, nanoparticle distribution, and clinical translation. The research discussed in this report aims to assist the effectiveness of MFH by directly addressing the issue of temperature control and nanoparticle distribution, which will ultimately help the clinical translation and further improve MFH treatment as a whole.

Temperature control and particle distribution will be addressed by solving Pennes' bioheat equation across irregular domains. For context, Pennes' bioheat equation is a fundamental mathematical model used to describe the distribution of heat in biological tissues, particularly in the context of hyperthermia. Developed by R. B. Pennes in 1948 [1], this equation considers both the conduction of heat within tissues and the heat transfer due to blood perfusion. The equation takes into account several key factors: the metabolic heat generation rate within tissues, the thermal conductivity of the tissue, the blood perfusion rate, and the specific heat capacity of the tissue. It describes how these factors interact to influence the temperature distribution within a biological tissue over time when subjected to an external heat source.

Pennes' bioheat equation is a partial differential equation and is often solved using numerical methods due to its complexity. The research in this paper introduces a new and improved numerical method to solve Eqn (1) across irregular domains. We introduce a new method called the *Augmented Matched Interface and Boundary*. There are many precursor steps required before being able to run the solver, including mathematically reconstructing any given irregular domain. Once the domain has been modelled, the solver can be ran to solve the Heat equation. Once the AMIB solver has successfully produced an answer, the heat distribution of a given interface, or tumor, can be accurately modelled.

Pennes' bioheat equation:

$$\rho C \frac{\partial T}{\partial t} = (k \nabla^2 T) + h_b + h_m \quad (1)$$

Table 1: Parameter List

Parameter Name	Description
ρ	Tissue density
C	Tissue heat capacity
t	Time
T	The unknown temperature
k	Thermal conductivity
h_b	Rate of heat transfer from blood to tissue
h_m	Rate of tissue heat production

0.2 Research Approach

0.2.1 Order of Operation

The research in this report has four main portions:

1. Application Prototype
2. Interface Reconstruction
3. Numerical Treatments
4. Augmented Matched Interface and Boundary Method

The application prototype section of the research focuses on the practical implementation of the project, demonstrating how the research findings can be applied in real-world scenarios.

The interface reconstruction section explores a method for reconstructing interfaces in numerical simulations, with a focus on accuracy and efficiency.

The numerical treatments section investigates various numerical techniques used to solve complex mathematical problems.

The augmented matched interface and boundary section presents a novel approach to solving interface problems using finite difference methods. This section discusses the implementation of the solver and evaluates its performance.

Before delving into the specifics of each section, it is important to introduce and define key terms that will be used throughout the report. These terms will help the reader understand the concepts and methodologies discussed in the subsequent sections.

0.2.2 Glossary

In terms of the application, we utilize a common tool called *control points* to allow for the reconstruction of an irregular domain. We have defined many types of control points which are of significance. They abide by the following assumptions:

- Essential
 - Defines where two line segments are not differentiable.
 - Typically at the sharp edges of a given interface.
- Primary
 - Located where derivatives change sign.
- Secondary
 - Ensures consecutive CPs do not have the same angle.
 - Provides accuracy benefits when reconstructing an interface.
- Origin
 - Assists when determining aforementioned control point types.
 - Used in the app when converting of pixel coordinates to real coordinates
 - Must be inside the interface.

0.3 Application Prototype

0.3.1 Description

The application prototype was made using MATLAB. It initially allows the user to upload an image of an irregular domain. Once uploaded, the user is requested to mark control points on the surrounding border of the desired interface. From there, the app can automatically detect the control point types and export metadata for the next step in the process, which is interface reconstruction. The following table represent the output format of the app, where each row contains metadata about a control point:

Table 2: Application Output Format

ID	TYPE	X	Y
----	------	---	---

The **ID** is an auto-generated integer by the application to uniquely identify each row. The **TYPE** is the automatically detected control point type, represented by an integer. **X** & **Y** are the coordinates of a given control point, which take on decimal values.

In addition, there is an option to have both the exported x and exported y coordinates in a relative unit. This calculation is dictated by a distance defined line segment manually set by the user. For the prototype, we assume the distance is in centimeters (CM).

0.3.2 Usage

Figure 1: The overall view of the application prototype.

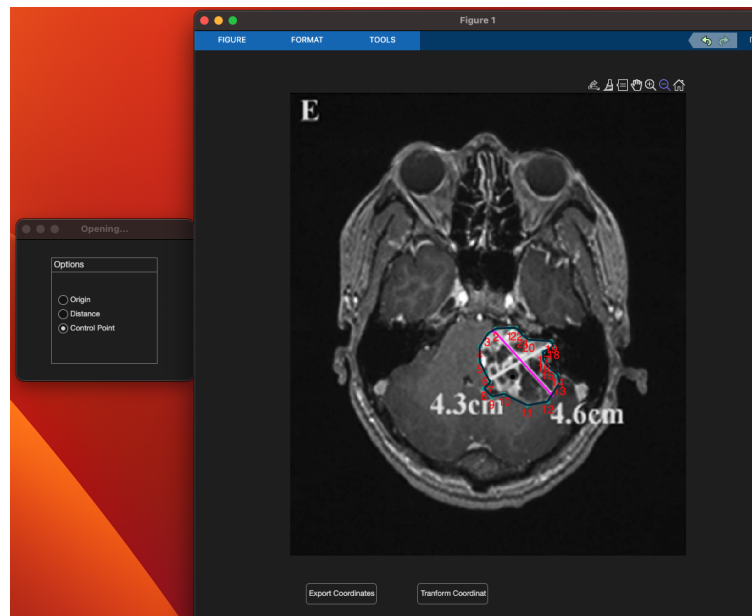


Figure 2: A zoomed-in view of the interface.

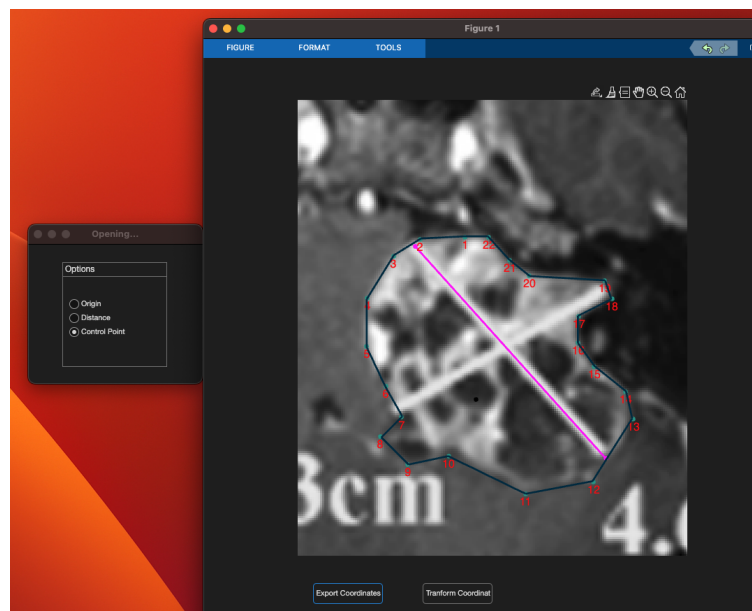
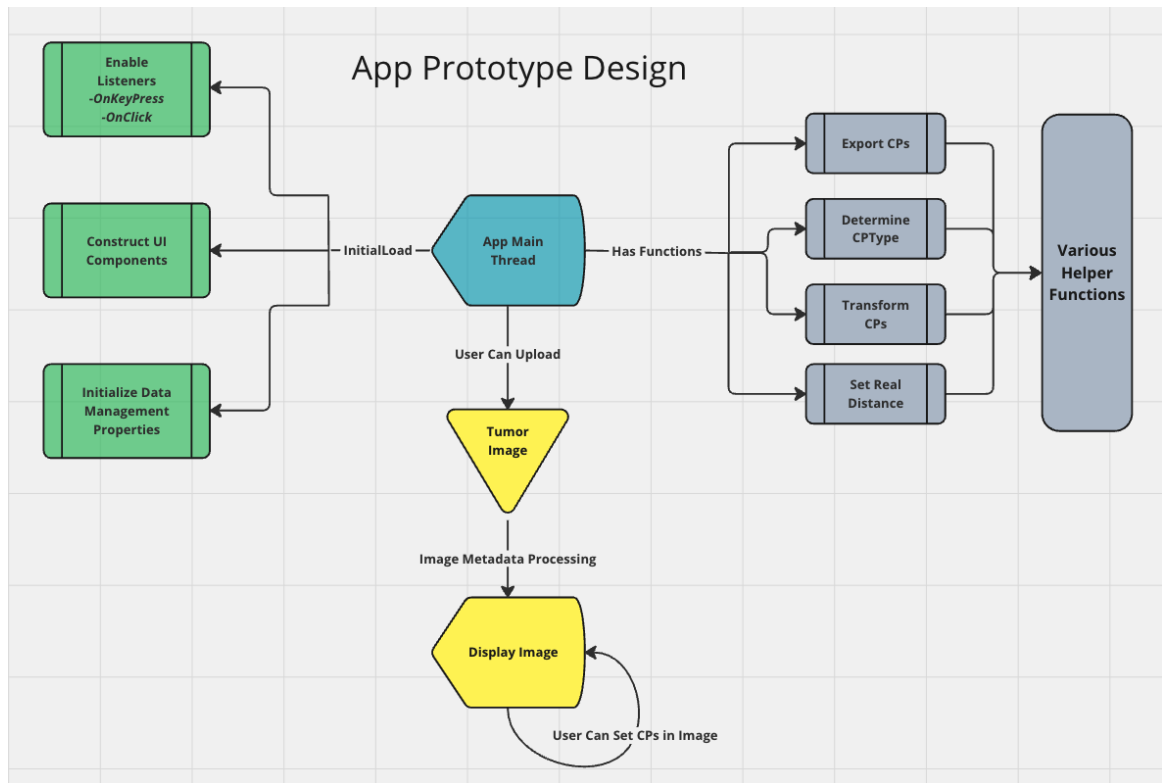


Figure 3: Sample output.

!	id	type	x	y	unit = cm	!
	1	2	12.1322	12.6040		
	1	3	11.7615	12.6040		
	2	3	11.0538	12.6377		
	3	2	10.6157	12.9073		
	4	1	10.1775	13.6150		
	5	1	10.1775	14.3901		
	6	3	10.4809	15.0304		
	7	1	10.7505	15.5022		
	8	1	10.4135	15.8392		
	9	2	10.8516	16.2773		
	10	3	11.4919	16.1425		
	11	3	12.7388	16.7491		
	12	2	13.8172	16.5469		
	13	1	14.4575	15.5359		
	14	1	14.3564	15.0978		
	15	3	13.8509	14.6934		
	16	1	13.5813	14.3227		
	17	1	13.5813	13.8846		
	18	1	14.1205	13.6150		
	19	1	14.0194	13.3117		
	20	3	12.8062	13.2443		
	21	3	12.4692	12.9747		
	22	2	12.1322	12.6040		

Figure 4: The visualized software architecture.



0.4 Interface Reconstruction

0.4.1 Natural Cubic Spline Interpolation

Before the Heat equation can be solved, we need to be able to mathematically model the irregular interface. To accomplish this, natural cubic spline interpolation (NCSI) was used. NCSI is a method used to insert a smooth curve in a set of data points while ensuring there are no sharp bends. It involves fitting a piece-wise cubic polynomial to the data, with the added constraint that the second derivative is continuous at each data point. We create a cubic polynomial between each consecutive control point. Therefore, the amount of piece-wise defined cubic polynomials created for the interpolation is dependent upon how many control points we have. The main reasons we utilized NCSI is because of the following guarantees:

1. Interpolation

- This allows us to ensure the the curve passes through each control point

2. Continuity

- First and second derivatives of adjacent segments are equal at each interior point

$$S'_{i-1}(x_i) = S'_i(x_i)$$

$$S''_{i-1}(x_i) = S''_i(x_i)$$

3. Smoothness

- Guarantees the curve to be differentiable
- This is required for downstream calculations

4. Natural Boundary

- The second derivative is set to zero at the endpoints
- Ensuring a "natural" behavior at the boundaries

The general outline of solving NCSI is as follows, given a set of cubic polynomials:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (2)$$

where a_i , b_i , c_i , and d_i are coefficients specific to each segment.

From there, construct a system of equations to find the coefficients of the cubic polynomials. The system typically is tridiagonal as shown below:

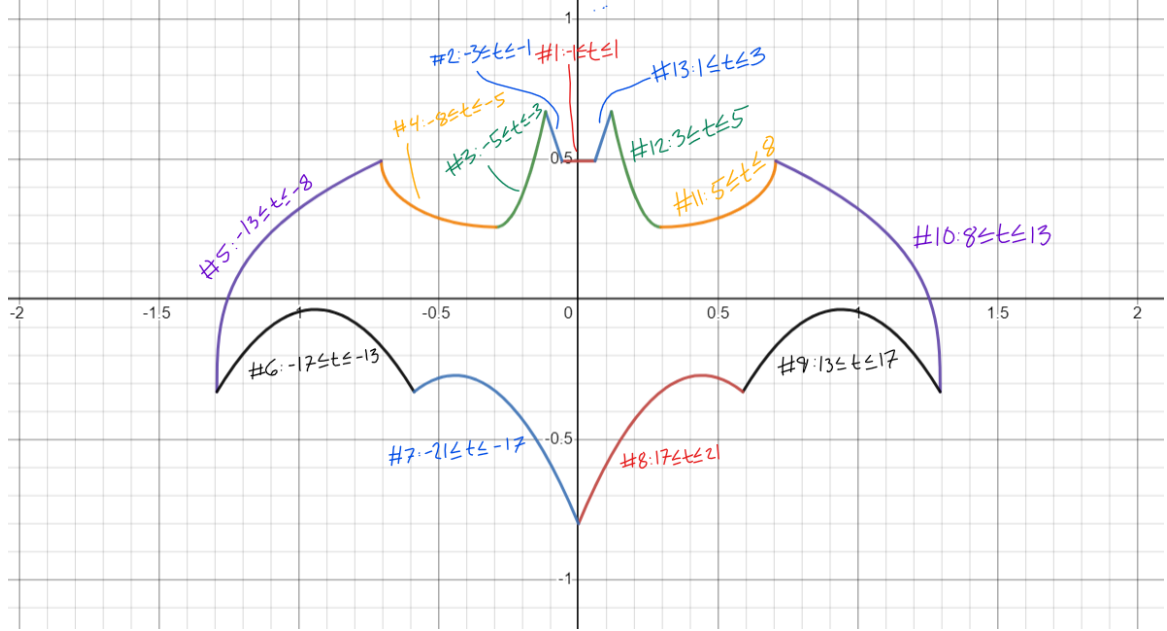
$$\begin{bmatrix} b_1 & c_1 & 0 & \cdots & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & & \vdots \\ 0 & a_3 & b_3 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & c_{n-1} & 0 \\ \vdots & & \ddots & a_{n-1} & b_{n-1} & c_{n-1} \\ 0 & \cdots & \cdots & 0 & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_{n-1} \\ d_n \end{bmatrix} \quad (3)$$

At this point, any means of solving a system of equations can be utilized to produce the values needed for each segment of the cubic spline interpolation.

0.4.2 Results

To produce benchmarks of how our implementation of NCSI performs, we first start with a piece-wise defined interface. The batman interface is shown below:

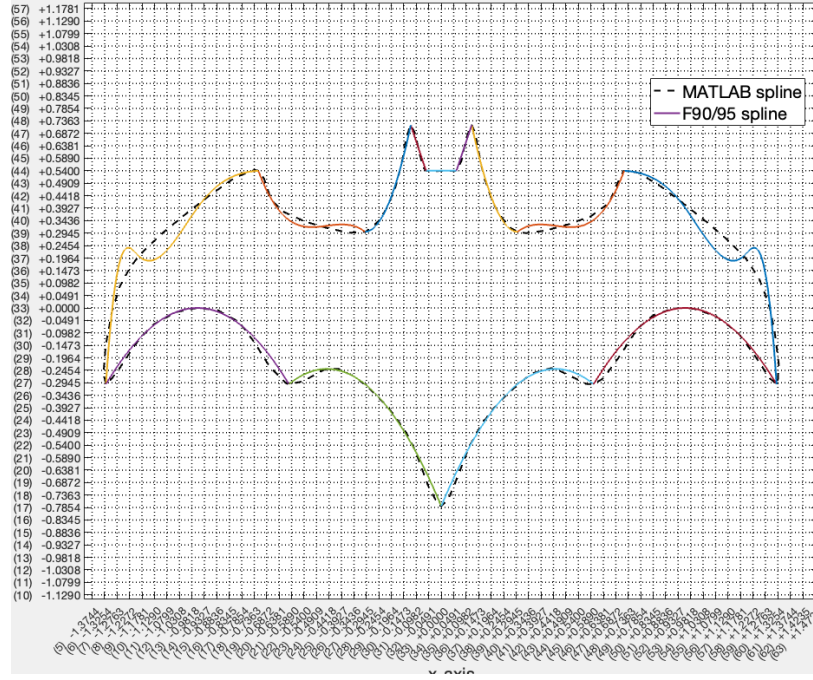
Figure 5: **Piece-wise defined batman interface.**



This approach is desirable because the exact answer for each segment is known.

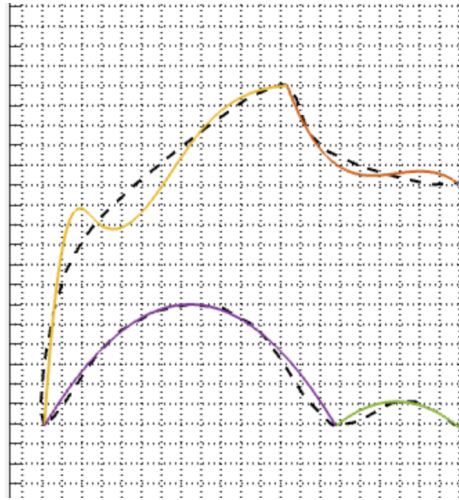
This allows us to compare the results of the NCSI implementation to the exact answer of what the interface should be. The visualized results are shown on the next page.

Figure 6: NCSI compared to exact solution



As we can see, the NCSI calculations are close to the original interface definition, but there are inaccuracies. Consider the zoomed-in view below:

Figure 7: NCSI Inaccuracy

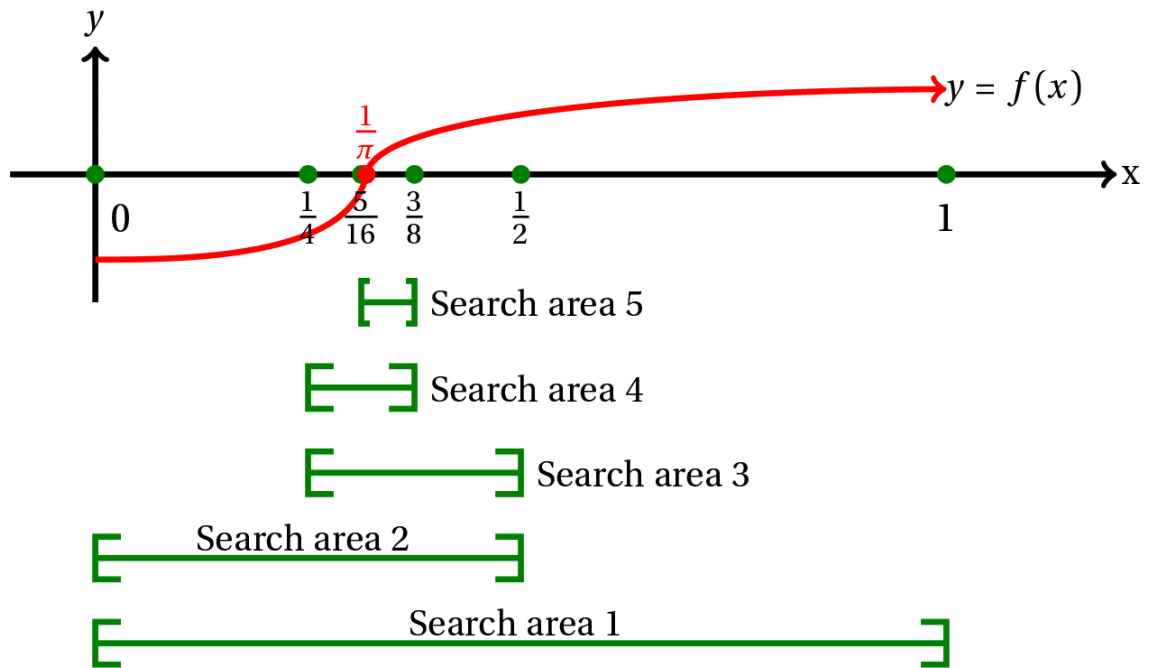


The findings show that our implementation of NCSI is not up to the required accuracy standards. Having an inaccurate reconstruction of an interface will produce inaccurate results when solving the heat equation. To alleviate this, a parametric cubic spline algorithm has the potential to produce more accurate results.

0.5 Five Numerical Treatments

In each section there has been valuable metadata which needs to be collected for the subsequent step(s) to function properly and accurately. In this section, we focus on many key questions which the AMIB solver needs to utilize when solving the heat equation. Each subsection will go into the question, pseudo-code, and experimental results. It's important to note that each of the following treatments utilize a modified form of the bisection method (Refer to figure 8 for visual representation). This search method was chosen because of it's simplicity. If the treatments can produce accurate results using a primitive searching method, then there is room for future performance increases.

Figure 8: **Bisection Method Visualized**



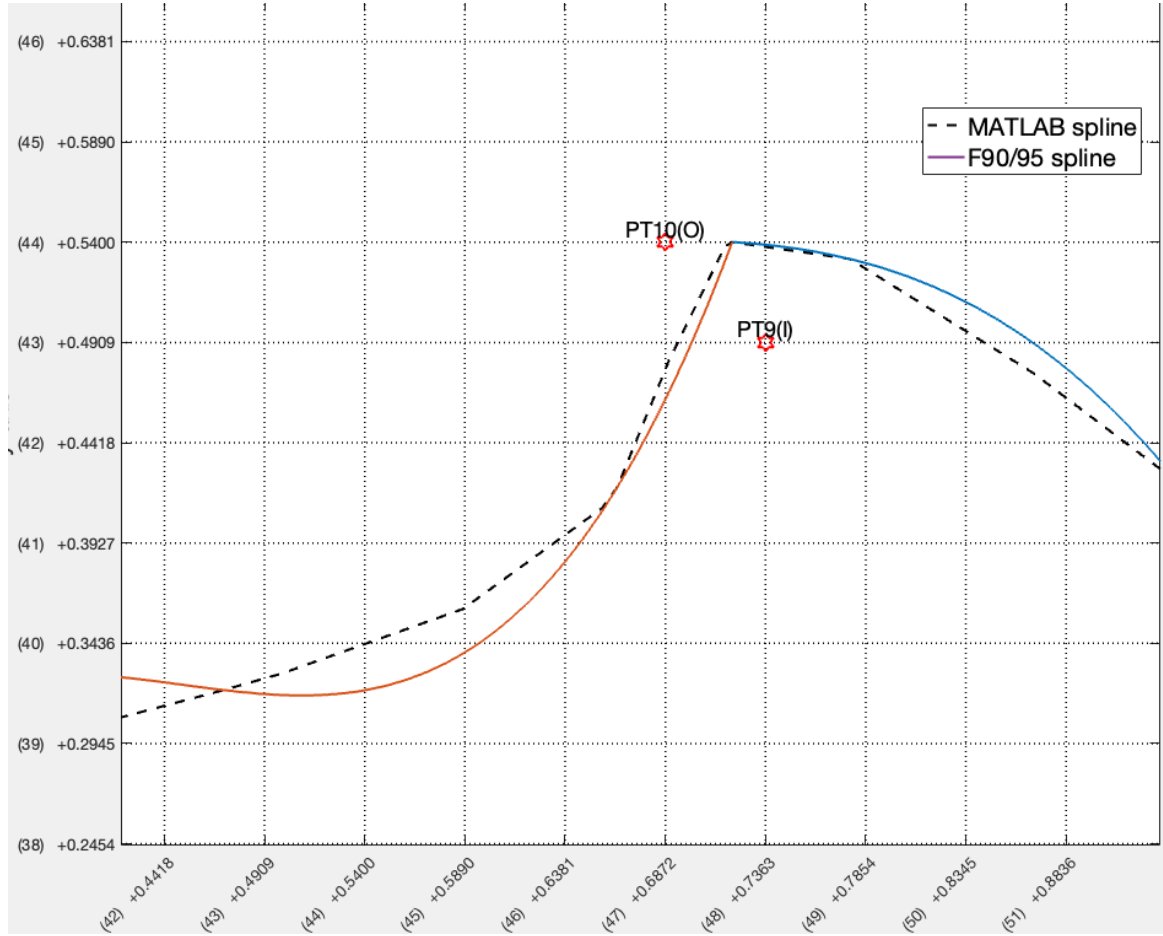
0.5.1 SetS

Given a point $P(x_o, y_o)$, how do we determine if P is inside or outside the interface?

1. Connect $P(x_o, y_o)$ to the origin
2. Set count = 0
 - This stores the number of intersections between the interface and line connecting P to the origin.
3. Find consecutive CPs c_i and c_{i+1} who have angles s.t. $\theta_i < \theta_o < \theta_{i+1}$, where θ_o is $P(x_o, y_o)$'s angle.
4. Use bisection method on t_i and t_{i+1} to find the t-value(s) of the intersection(s):
 - $t_{i+\frac{1}{2}} = \frac{t_i + t_{i+1}}{2}$
 - Plug in calculated $t_{i+\frac{1}{2}}$ into the $x(t)$ and $y(t)$ pieces
 - Get the angle of the midpoint, $\theta_{i+\frac{1}{2}}$
 - Repeat until $\left| \theta_{i+\frac{1}{2}} - \theta_o \right| < \text{TOL}$
5. If the intersection is closer to the origin than P, add 1 to count
6. After every pair of CPs is checked, inspect the count
 - If count is even, then P is inside the interface.
 - If count is odd, then P is outside the interface.

The screenshot below highlights the results of the implementation.

Figure 9: SetS Results Visualized



It is important to note that PT10 is labelled **O** which represents the SetS method detected the grid point was outside the interface. PT9 is labelled **I** which represent the SetS method detected the grid point was inside the interface. This method looks to be producing accurate results.

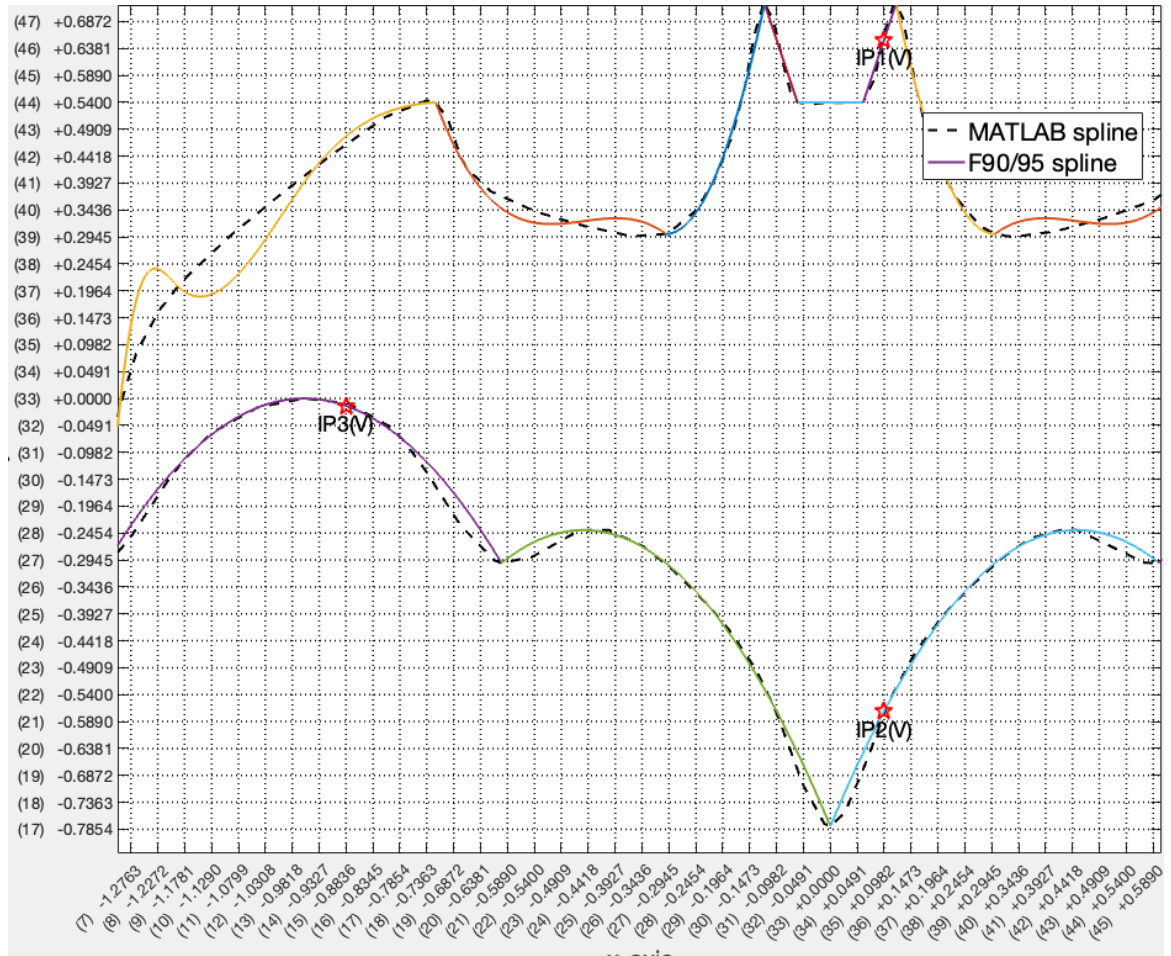
0.5.2 GammaX

Given the coordinate of a gridline, how do we determine where the interface intersects the gridline?

1. Given y_i (the y-coord of a gridline) and endpoints CP_1 and CP_2 , find the x-coord of the intersection.
2. Let iteration count = 0
3. Store parameter values of CP_1 and CP_2 as t_1 and t_2
4. Let $y_1 = CP_1$'s y-coord and $y_2 = CP_2$'s y-coord.
5. Do until the max # of iterations is reached or forcefully exited
 - Let $t_m = \text{average of } (t_1, t_2)$.
 - Find the coords, (x_m, y_m) at point t_m
 - Let $x = x_m$
 - Exit if y_m is sufficiently close to y_i
 - If y_m is sufficiently close to CP_1 's y-coord, then $x = CP_1$'s x coord, and exit.
 - If y_m is sufficiently close to CP_2 's y-coord, then $x = CP_2$'s x coord, and exit.
 - Get the coords at t_1 and t_2 , (x_1, y_1) and (x_2, y_2)
 - Increment the iteration counter
 - If $y_m > y_j > y_2$ or $y_m < y_j < y_2$, then set $t_1 = t_m$
 - If $y_m > y_j > y_1$ or $y_m < y_j < y_1$, then set $t_2 = t_m$

The screenshot below highlights the accuracy of this function:

Figure 10: **GammaX Results Visualized**



Notice how each star is located on a y-intersection grid point for a given location on the interface.

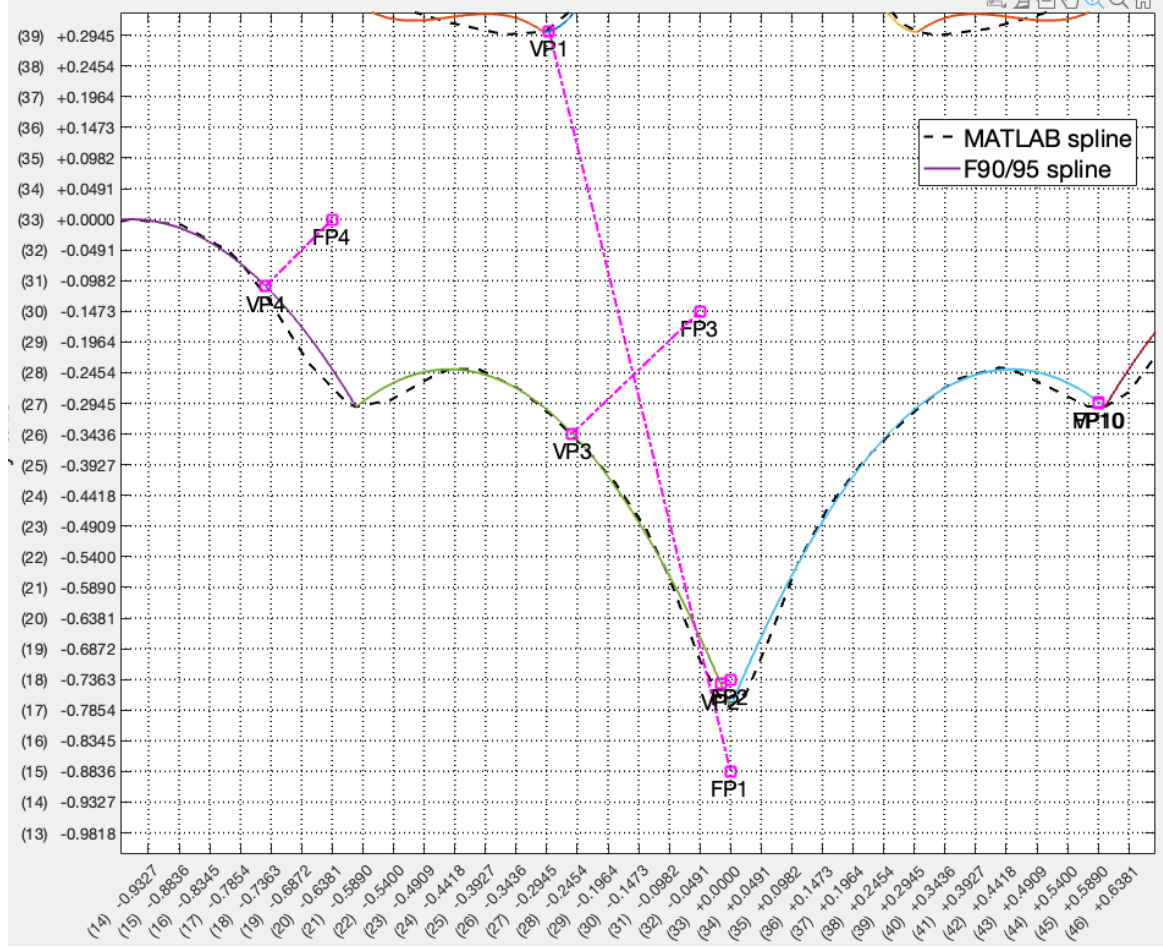
0.5.3 SeekVP

Given $P(x_0, y_0)$, find $Q(x_n, y_n)$ such that $\vec{PQ}$ is $\perp$ at Q ?

1. Given i_x and i_y , find the coords of the mesh point $M(x_m, y_m)$.
2. For each pair of consecutive CPs, CP_i and CP_{i+1}
 - (a) Calculate θ for CP_i and CP_{i+1} , denoted by θ_i and θ_{i+1}
 - (b) Modify angles such that $0 \leq \theta_i, \theta_{i+1} < 2\pi$
 - (c) If one of θ_i and θ_{i+1} is in quadrant 3, and the other is in quadrant 4
 - Modify θ_i and θ_{i+1} s.t. one is in quadrant 1 and the other is in quadrant 2.
 - (d) If one of θ_i and θ_{i+1} is in quadrant 1 and the other is in quadrant 2, and CP_i and CP_{i+1} 's parameters values are different
 - If $\theta_i < \frac{\pi}{2}$, call bisection method with CP_i as the first endpoint and CP_{i+1} as the second endpoint
 - If $\theta_{i+1} < \frac{\pi}{2}$, call bisection method with CP_i as the second endpoint and CP_{i+1} as the first endpoint
 - (e) If bisection is successful,
 - Store value of the found VP
 - Find coordinates of found VP, $P(x_0, y_0)$
 - Calculate distance from P to M
 - If first VP is found
 - Store VP coords as Q
 - Record the distance from Q to M
 - If distance from P to M is $<$ the distance from Q to M
 - Update Q to contain P's coords
 - Update the distance from Q to M
3. If no suitable VP is found, record failure and exit
4. Else, return coords of Q

The screenshot below highlights the accuracy of this function:

Figure 11: SeekVP Results Visualized



Notice how the lines from $FP4 \rightarrow VP4$, $FP3 \rightarrow VP3$, and $FP1 \rightarrow VP1$ are all perpendicular to the location on the interface. The output of this method is accurate. It's important to highlight the behavior of the perpendicular point found when starting the algorithm from $FP1$. In this case, the found point $VP1$ is on the opposite end of the interface; however, the method is still accurate. This poses an interesting case and is to be noted when solving the heat equation during later steps.

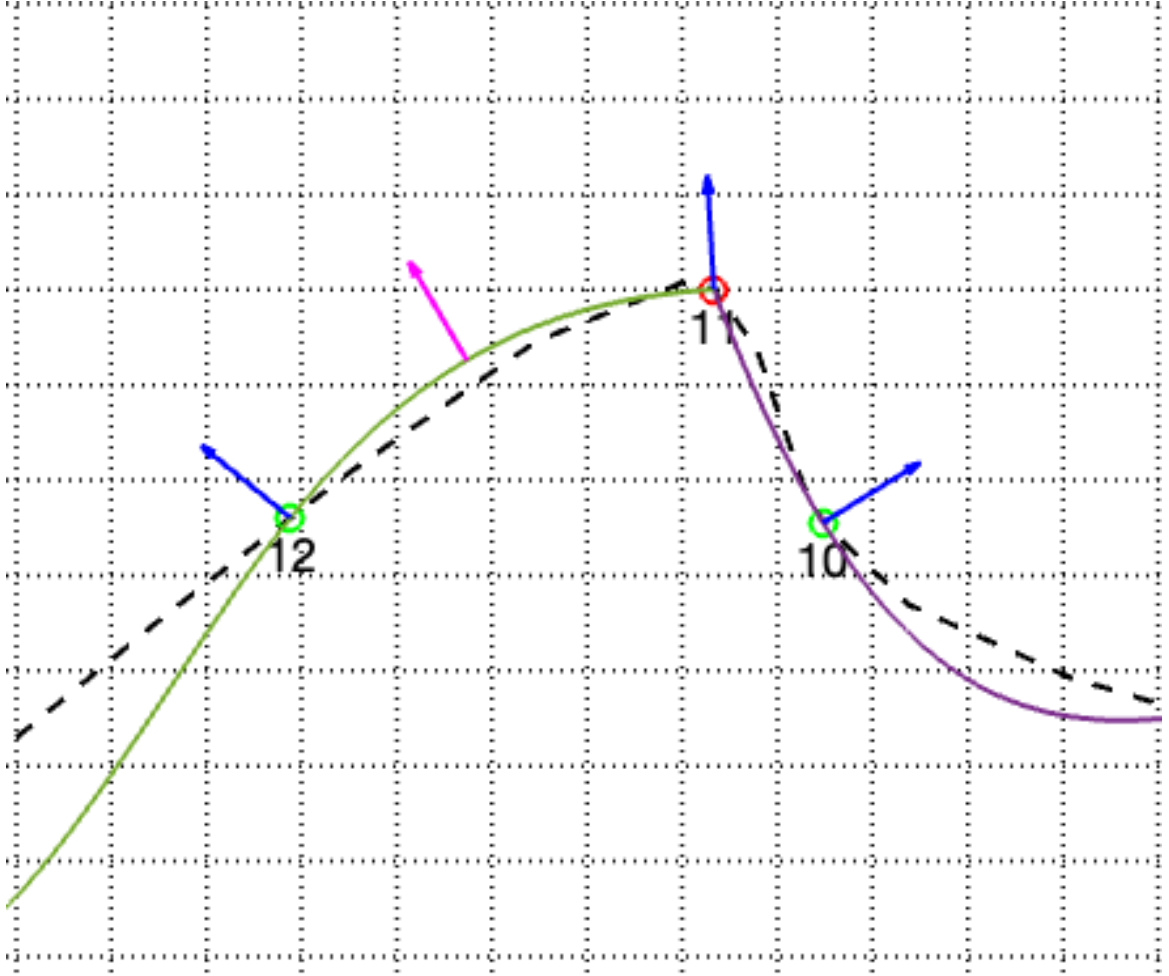
0.5.4 GetNormal

How do we find the normal direction for a given CP?

1. Given $Q(x_n, y_n)$, calculate the angle from the positive x-direction to the line connecting from the origin to Q. Call this angle Q's xop angle.
2. Add 2π to the xop angle of the last CP in the list.
3. For each pair of consecutive CPs, CP_i and CP_{i+1} :
 - (a) If Q is sufficiently close to CP_i
 - θ is the desired angle for CP_i .
 - Undo the special case in (2)
 - Return θ
 - (b) If Q is sufficiently close to CP_{i+1}
 - θ is the desired angle for CP_{i+1}
 - Undo the special case in (2)
 - Return θ
 - (c) If x_n is between the x values of CP_i and CP_{i+1} , and y_n is between the y values of CP_i and CP_{i+1}
 - Use bisection on Q's xop angle, CP_i and CP_{i+1} to find the parameter value, t_m , where the line from the origin to Q intersects the interface
 - Using t_m and the FID from CP_i , calculate θ value at Q
 - Undo special case
 - Return θ

The screenshot below highlights the accuracy of this function:

Figure 12: **GetNormal Results Visualized**



In the above figure, the blue lines represent the normal direction at a specific type of control point. It is clear the calculated outer normal direction is correct. The GetNormal method is robust in the sense that it is not limited to control point coordinates. Notice the pink line; this represents the outer normal direction for an arbitrary point on the interface. GetNormal produces accurate results when calculating the normal direction for any location on the interface. At this point, all required metadata has been gathered and can be utilized in the next section, the AMIB numerical method.

0.6 Method

0.6.1 Mathematical Models

The Augmented Matched Interface and Boundary (AMIB) method aims to solve two key mathematical models. The first of these mathematical models is the Poisson equation (eqn. 4). A time-independent equation, the Poisson equation is used to understand the distribution of heat within a given domain at a specific instance of time.

$$\Delta u + ku = f(\vec{x}), \quad (4)$$

The Poisson equation is paired with a boundary condition (eqn. 5) to make a Boundary Value Problem (BVP).

$$\alpha_{\Gamma}u + \beta_{\Gamma}\frac{\partial u}{\partial n} = \phi(\vec{x}), \quad (5)$$

An example of solving solving a BVP (Poisson equation) is show in Figure 13.

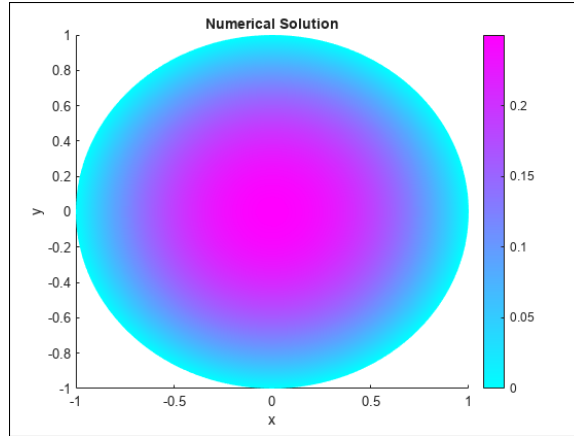


Figure 13: Numerical solution from solving the Poisson equation on a two-dimensional disk-shaped interface.

The second mathematical model is the Heat equation (eqn. 6): A time-dependent equation used to understand how heat spreads within a given domain over a period of time. Essentially, the Heat equation is made up of a Poisson equation at each step of time.

$$\frac{\partial u}{\partial t} = \beta \Delta u + g, \quad 0 \leq t \leq T, \quad (6)$$

The Heat equation is paired with a boundary condition (eqn. 7)

$$\alpha_{\Gamma}u + \beta_{\Gamma}\frac{\partial u}{\partial n} = \psi(t, \vec{x}), \text{ on } \Gamma, \quad (7)$$

and an initial condition (eqn. 8) to make an Initial and Boundary Value Problem (IBVP).

$$u(0, \vec{x}) = u_0(\vec{x}). \quad (8)$$

For both the BVP and IBVP, a variety of unique boundary conditions can be achieved by changing the α_{Γ} and β_{Γ} parameters. For example, eqns. 5 and 7 become a Dirichlet boundary condition when $\alpha_{\Gamma} \neq 0$ and $\beta_{\Gamma} = 0$, a Neumann boundary condition when $\alpha_{\Gamma} = 0$ and $\beta_{\Gamma} \neq 0$, and a Robin boundary condition when $\alpha_{\Gamma} \neq 0$ and $\beta_{\Gamma} \neq 0$.

To numerically solve the BVP or the IBVP, we immerse the BVP or IBVP in a rectangular domain and then solve the similar immersed problem: The Immersed BVP (ImBVP) or the Immersed IBVP (ImIBVP). An example of setting up an immersed problem is shown in Figure 14.

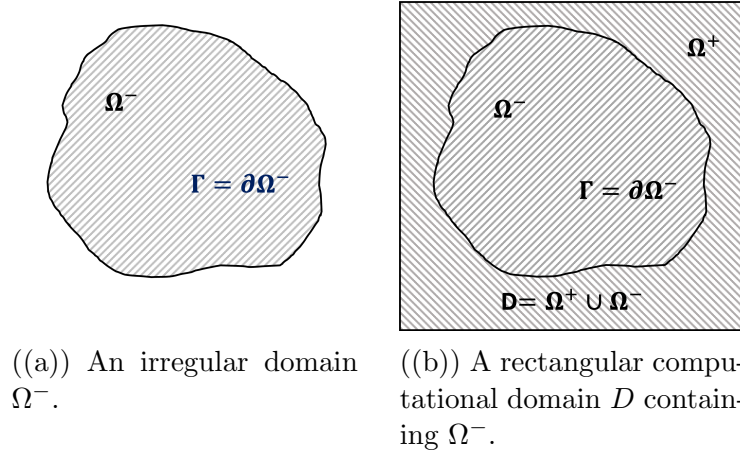


Figure 14: A demonstration of the original problem and its respective immersed problem.

0.6.2 Model Applications

The Poisson-Boltzmann, a type of Poisson equation, is used to understand the electrostatic potential distribution over a protein as shown in Figure 15(a).

Whereas the Pennes Bioheat equation, a type of Heat equation, is used to understand the heat dissipation in Magnetic Fluid Hyperthermia to remove a cancerous tumor like the one shown in Figure 15(b).

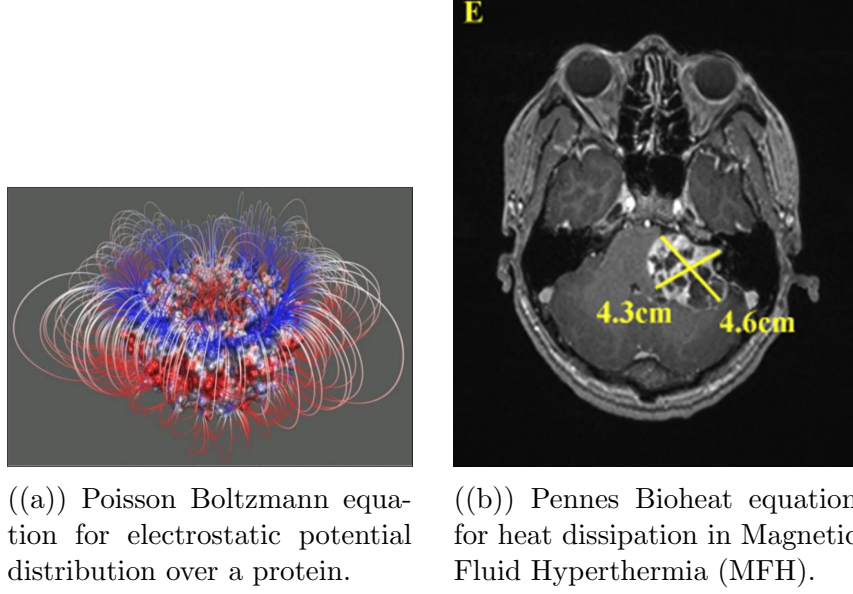


Figure 15: Applications of the Poisson eqn. and Heat eqn..

0.6.3 AMIB Method

The Augmented Matched Interface and Boundary (AMIB) method aims to numerically solve ImBVPs and ImIBVPs on highly irregular, smooth, two-dimensional domains while maintaining fourth-order accuracy in space and second-order accuracy in time. The three key steps for creating the AMIB method are described in the following three subsections, though significant detail of the formulation of the AMIB method is skipped for the sake of brevity.

0.6.4 Interface Points, Fictitious Points, and Vertical Points

The first crucial step to formulate the AMIB method is to identify three types of points: Interface Points, Fictitious Points, and Vertical Points. To demonstrate the three types of points, we introduce the butterfly-shaped interface shown in Figure 16. The butterfly-shaped interface is a highly-irregular, yet smooth two-dimensional interface. Three different colors of points are noticeable when analyzing the zoomed-in region of the interface as shown in Figure 17. Interface

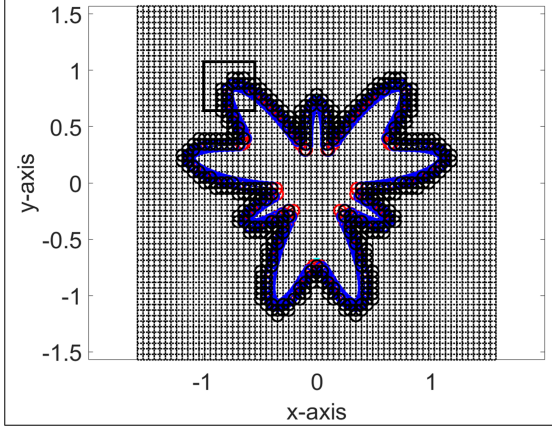


Figure 16: Full-view of the butterfly-shaped interface.

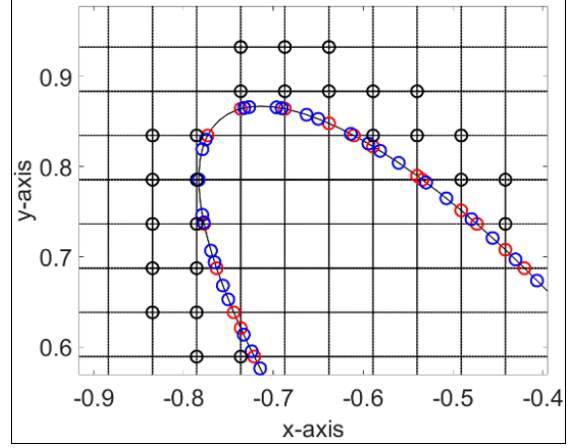


Figure 17: Zoomed-in region of the butterfly-shaped interface.

points (IPs), demonstrated by the red circles, are points located on the interface where the interface Γ intersects with a horizontal or vertical grid line. Interface points are used to roughly determine the shape of the interface and are found via Newton's method. Next, fictitious points (FPs), demonstrated by the black circles, are located at the grid points located in the Ω_+ subdomain within the first two layers of grid lines from the interface. Fictitious points are a significant part of the identity and success of the AMIB method. Each fictitious point has an associated fictitious value (FV). The fictitious value represents the value of the unknown solution u at the fictitious point if the solution spread from inside the subdomain Ω_- to the fictitious point. Fictitious points are necessary to maintain fourth-order approximation in space when solving on irregularly-shaped interfaces. Specifically, fictitious points are necessary to approximate the Laplacian in eqns. 4 and 6. Lastly, vertical points (VPs) are demonstrated by the blue circles. Each fictitious point has a paired vertical point such that the line passing from the fictitious point to its paired vertical point is perpendicular to the interface at the vertical point. Vertical points, found via a Newton's iteration method, are necessary because the interface conditions (eqns. 5 and 7) at the VPs are used to construct the FV representations at the FPs. After all IPs, FPs, and VPs have been identified, the next key step in the AMIB method is to construct the FV representations.

0.6.5 Fictitious Value Representations at Fictitious Points

Most fictitious value representations can be constructed using eqn. 9. In both Figures 18 and 19, the black square represents the fictitious point where we desire to construct the fictitious value $\tilde{u}_{\text{FP}}$, the circles with a ring around them represent vertical points from the set of vertical points V_{FP} , and the circles with no surrounding rings represent chosen grid points from the set of chosen grid points S_{FP} . The color associated with the VPs and chosen grid points on the graph represent the weight associated with that VP or chosen grid point. The weight corresponds to how much the value of the VP or chosen grid point is used when constructing the FV representation. Figure 18 shows a relatively simple construction of the FV representation at the fictitious point on the graph by using only one VP. Whereas Figure 19 show a more complicated construction of the FV representation due to requiring multiple VPs. The formulation for the construction of fictitious values is given in eqn. 9:

$$\tilde{u}_{\text{FP}} = \sum_{(x_I, y_J) \in S_{\text{FP}}} \tilde{w}_{\text{I,J}} u_{\text{I,J}} + \sum_{\vec{x}_{\text{VP}_i} \in V_{\text{FP}}} \tilde{w}_{\text{VP}_i} \phi(\vec{x}_{\text{VP}_i}), \quad (9)$$

where S_{FP} is a set of chosen grid points, V_{FP} is a set of vertical points, and w represents the weight associated with the specified point. Once as many FV representations have been constructed as is possible, the AMIB method iteratively uses the known FV representation constructions to find the unknown FV value representations constructions. Once a suitable number of FV representation constructions are known, the next major step in the AMIB method is to set up and solve the augmented system.

0.6.6 The Augmented System

The last step in the AMIB method is to set up and solve the augmented system. The augmented system is set up by combining eqns. 10 and 11. To aid in understanding the size of the formulation of the system, let N_1 = the number of interior grid points and N_2 = the number of interface points.

In eqn. 10, A is a $N_1 \times N_1$ matrix for solving the standard approximation, B is a $5N_2 \times N_1$ matrix consisting of correction term coefficients, and F is a $N_1 \times 1$

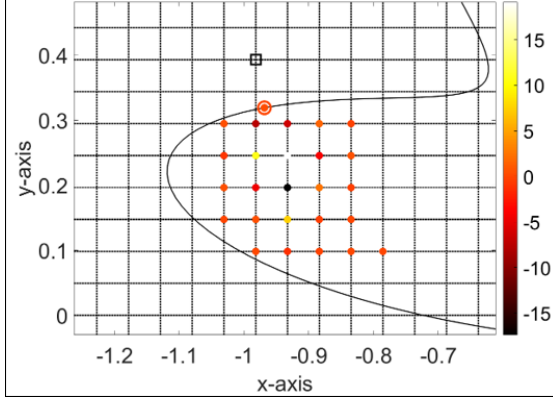


Figure 18: A relatively simple FV representation construction.

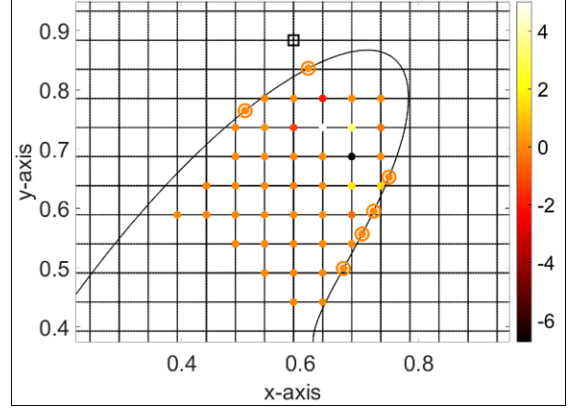


Figure 19: A more complicated FV representation construction.

vector consisting of the values of the known function f at each interior grid point. U and Q are both unknown vectors that we aim to solve. Specifically, U is a vector of dimension $N_1 \times 1$ that consists of the unknown values of u at all interior grid points, and Q is a vector of dimension $5N_2 \times 1$ consisting of correction terms on the interface. The nonzero entries of matrix B are shown in Figure 20.

$$AU + BQ = F \quad (10)$$

In eqn. 11, C is a $5N_2 \times N_1$ matrix consisting of weights and products of weights, I is the identity matrix of dimension $5N_2 \times 5N_2$, and Φ is a $5N_2 \times 1$ vector consisting of the known interface values. The nonzero entries of matrix C are shown in Figure 21.

$$CU + IQ = \Phi. \quad (11)$$

The augmented system is constructed by combining eqns. 10 and 11 to create the system as shown in eqn. 12.

$$\begin{pmatrix} A & B \\ C & I \end{pmatrix} \begin{pmatrix} U \\ Q \end{pmatrix} = \begin{pmatrix} F \\ \Phi \end{pmatrix}. \quad (12)$$

The augmented system is massive and requires significant care to solve efficiently. In the simplest case, where the ImBVP or ImIBVP is solved using the most coarse mesh size (mesh size means the space between consecutive grid lines) of 65×65 , matrix A contains over 17 million entries and the system as a whole

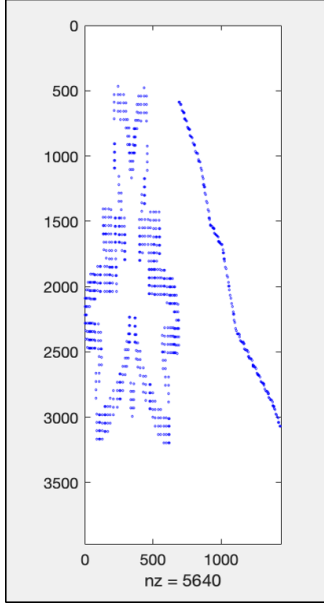


Figure 20: Nonzero entries in matrix B .

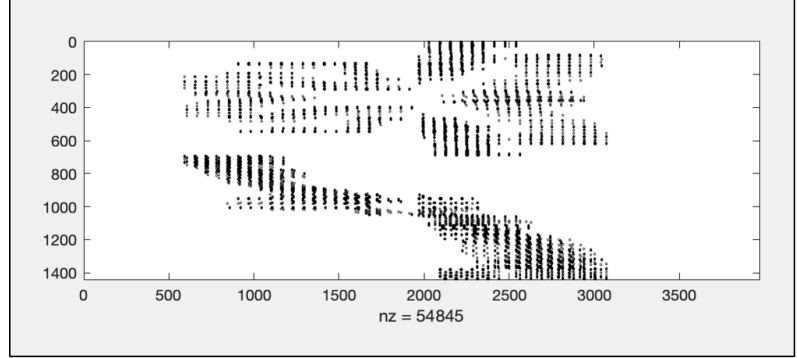


Figure 21: Nonzero entries in matrix C .

contains approximately 32 million entries. It is important to note that the majority of the entries are zero in the system, but the system as a whole is nonetheless massive even when using the coarsest mesh size. Due to the size of the system, and specifically the size of matrix A , the Fast Fourier Transform (FFT) is used to make the solving of the augmented system highly efficient as detailed in [2]. Finally, the system is solved using the Schur complement method as described in [3].

0.7 Results

Numerically solving an immersed problem like an ImBVP or ImIBVP involves a numerical error. The numerical error is the absolute difference between the exact solution and the numerical solution. At every single grid point, a numerical error may exist. As stated in the section introducing the AMIB method, the goal of the AMIB method is to numerically solve ImBVPs and ImIBVPs on highly irregular, smooth, two-dimensional domains while maintaining fourth-order accuracy in space and second-order accuracy in time. Thus, the aim of this section of the paper is to test the AMIB method and analyze the results of the tests in order to determine if the method is capable of achieving its goal.

For the first test, an ImBVP was solved on a starfish-shaped interface to

solve the Poisson equation. The numerical solution for the first test is shown in Figure 22, and the numerical results are shown in Table 3. Analyzing the contents of the table, it is clear that the AMIB method achieved its goal of fourth-order accuracy in space when solving for the Poisson equation on a highly irregular and smooth two-dimension interface like the starfish-shaped interface. Fourth-order accuracy means that as the number of grid lines are doubled, the numerical error decreases by $\frac{1}{2^4}$. The results from Table 3 show that the AMIB method achieves this fourth-order approximation for both the L^2 and L^∞ errors. The L^2 error is the average of the numerical errors at all interior grid points, whereas the L^∞ error is the maximum of the numerical errors at all interior grid points. Notably, the AMIB method achieves fourth-order accuracy for all tested mesh sizes as well and maintains a relatively low Bi-Conjugate Gradient (BCG) iteration number, highlighting the efficiency of the AMIB method.

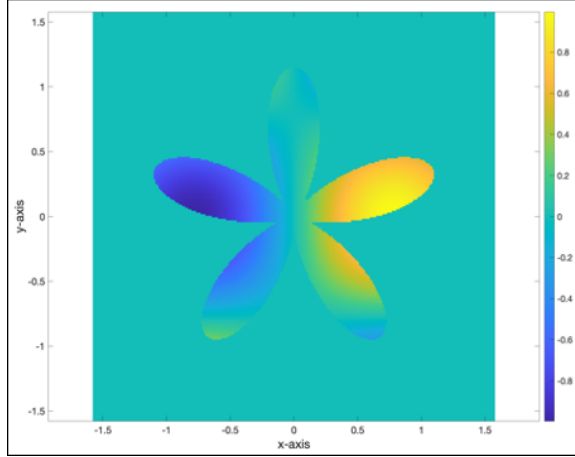


Figure 22: Numerical solution of the "starfish" interface.

Table 3: Convergence tests for solving the ImBVP with the "starfish"-shaped interface, Test 1

$[n_x, n_y]$	L^∞		L^2		BCG iter no.
	error	order	error	order	
[65, 65]	1.91E-06		6.11E-07		37
[129, 129]	1.19E-07	4.00	4.55E-08	3.75	44
[257, 257]	5.01E-09	4.57	9.94E-10	5.52	47
[513, 513]	2.86E-10	4.13	5.63E-11	4.14	51

For the second test, both an ImBVP and ImIBVP were solved on a

butterfly-shaped interface to solve the Poisson and Heat equations. The numerical solution for the second test is shown in Figure 23, and the numerical results are shown in Tables 4 and 5. Analyzing Table 4, solving the ImBVP and ImIBVP on the butterfly-shaped interface once again results in fourth-order accuracy in the space domain for both the L^2 and L^∞ errors on all tested mesh sizes.

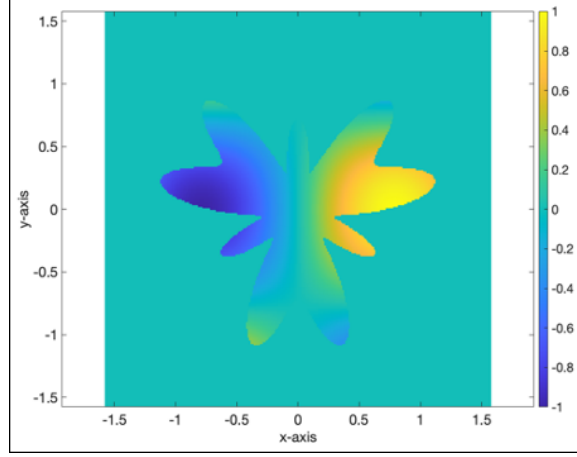


Figure 23: Numerical solution of the "butterfly" interface.

Table 4: Spatial convergence tests for solving the ImBVP and ImIBVP with the "butterfly"-shaped interface, Test 2

ImBVP	L^∞		L^2		BCG
$[n_x, n_y]$	error	order	error	order	time (sec)
[65, 65]	1.15E-04		1.09E-05		28
[129, 129]	5.39E-07	7.74	1.24E-07	6.46	69
[257, 257]	5.50E-09	6.62	1.38E-09	6.48	293
[513, 513]	3.09E-10	4.15	1.02E-10	3.76	1351
ImIBVP	L^∞		L^2		BCG
$[n_x, n_y]$	error	order	error	order	time (sec)
[65, 65]	6.46E-05		5.39E-06		34
[129, 129]	8.59E-08	9.55	5.79E-09	9.86	123
[257, 257]	1.75E-09	5.61	1.52E-10	5.25	528
[513, 513]	1.02E-10	4.11	9.30E-12	4.03	1973

Since the Heat equation is a time-dependent equation, the accuracy of the AMIB method within the time domain must be considered. Analyzing the results from Table 5, it can be noted that the AMIB method achieved at least second-order accuracy in time for each consecutive time step for both the L^2 and L^∞ errors. Additionally, Table 5's results demonstrate the unconditional stability of the AMIB

method since the method was able to solve a time-dependent equation on a highly irregularly-shaped domain without the resulting numerical solution "blowing up".

Table 5: Temporal convergence tests for solving the ImIBVP with the "butterfly"-shaped interface, Test 2

n_t	L^∞		L^2		BCG time (sec)
	error	order	error	order	
2	1.67E-03		9.24E-04		82
4	4.01E-04	2.05	2.23E-04	2.05	160
8	9.99E-05	2.01	5.54E-05	2.01	308
16	2.49E-05	2.00	1.38E-05	2.00	568
32	6.23E-06	2.00	3.46E-06	2.00	1104
64	1.56E-06	2.00	8.65E-07	2.00	2038
128	3.89E-07	2.00	2.16E-07	2.00	3802

For the third test, both an ImBVP and ImIBVP were solved on an aircraft-shaped interface to solve the Poisson and Heat equations. The aircraft interface was also solved while the interface scaled in size from small to large. The numerical solution for the third test is shown in Figure 24, the numerical results are shown in Table 6, and the numerical errors on the interface for the scaling problem are shown in Figure 25. Analyzing the spacial domain portions of Table 6, solving the ImBVP and ImIBVP on the aircraft-shaped interface once again resulted in fourth-order accuracy in the space domain for both the L^2 and L^∞ errors on all tested mesh sizes. For the time domain, the AMIB method achieved at least second-order accuracy in time for each consecutive time step for both the L^2 and L^∞ errors.

Analyzing Figure 25, it can be noted that the numerical errors when solving the "aircraft"-shaped interface on various scale sizes remain relatively low. Moreover, the results of Table 7 show both small L^2 and L^∞ errors demonstrating that the AMIB method can maintain low numerical error when solving immersed problems on highly irregularly-shaped, smooth, scaling two-dimensional domains.

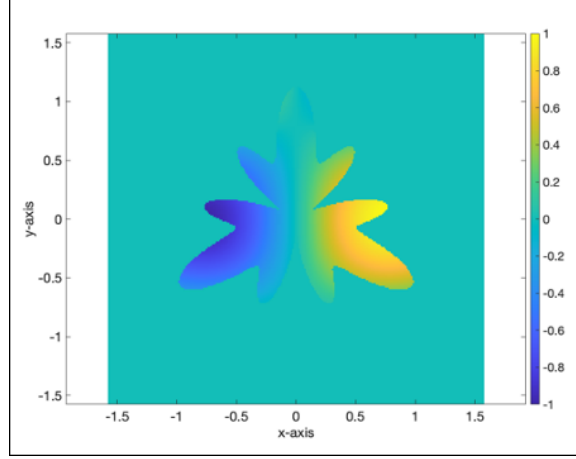


Figure 24: Numerical solution of the "aircraft" interface.

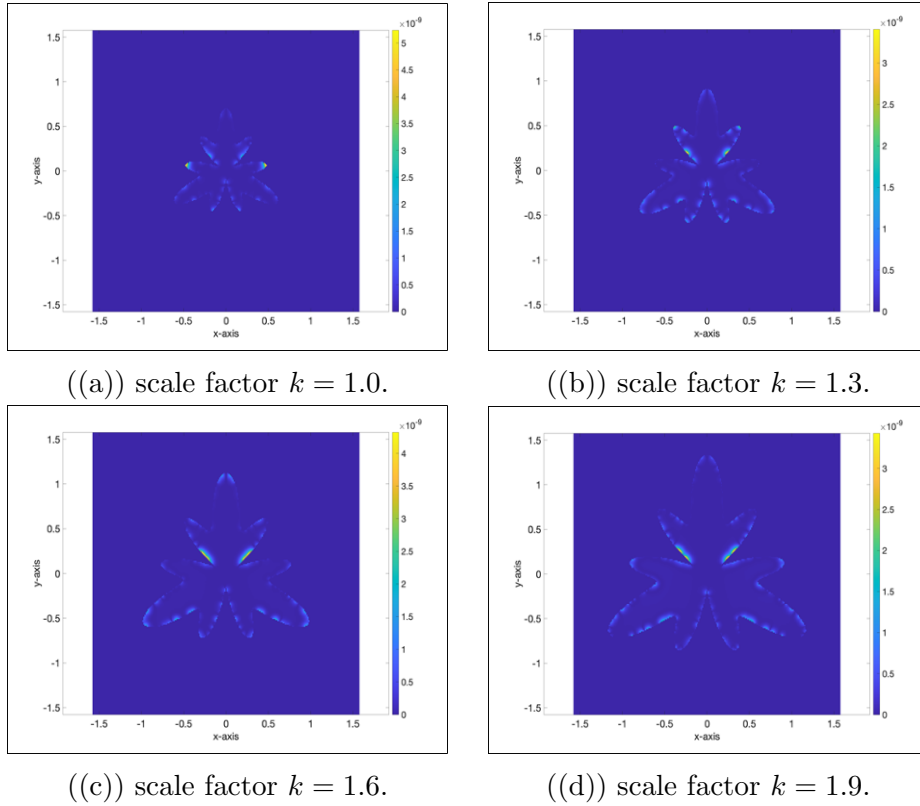


Figure 25: Numerical errors for solving the ImIBVP with the "aircraft"-shaped interface of various scale factors, Example 3.

0.8 Conclusion

This paper addressed the concept of solving the Heat equation across irregular domains. Firstly, the desired application of improving the cancer therapy treatment known as Magnetic Fluid Hyperthermia was introduced, and the intended application of improving MFH was related to a mathematical model, namely the

Table 6: Spatial and temporal convergence tests for solving the ImBVP and ImIBVP with the "aircraft"-shaped interface, Test 3

ImBVP					
$[n_x, n_y]$	L^∞		L^2		BCG time (sec)
	error	order	error	order	
[65, 65]	3.10E-05		5.60E-06		37
[129, 129]	2.05E-07	7.24	7.32E-08	6.26	75
[257, 257]	5.51E-09	5.12	1.60E-09	5.52	304
[513, 513]	2.90E-10	4.25	1.06E-10	3.92	1451

ImIBVP					
Spatial	L^∞		L^2		BCG time (sec)
$[n_x, n_y]$	error	order	error	order	
[65, 65]	5.74E-06		4.44E-07		35
[129, 129]	3.53E-08	7.34	3.92E-09	6.82	114
[257, 257]	1.56E-09	4.50	1.54E-10	4.67	532
[513, 513]	1.01E-10	3.96	1.00E-11	3.94	1935

Temporal	L^∞		L^2		BCG time (sec)
n_t	error	order	error	order	
2	1.45E-03		9.22E-04		95
4	3.51E-04	2.05	2.22E-04	2.05	179
8	8.74E-05	2.01	5.54E-05	2.01	334
16	2.18E-05	2.00	1.38E-05	2.00	633
32	5.45E-06	2.00	3.46E-06	2.00	1179
64	1.36E-06	2.00	8.64E-07	2.00	2171
128	3.41E-07	2.00	2.16E-07	2.00	3966

Table 7: Convergence tests for solving the ImIBVP with the "aircraft"-shaped interface of various scale factors, Test 3

scale factor k	no. of points		L^∞	L^2	BCG time (sec)
	IP	FP			
1.0	662	909	5.24E-09	3.78E-10	121
1.3	856	1198	3.41E-09	2.48E-10	122
1.6	1060	1479	4.32E-09	3.16E-10	141
1.9	1266	1765	3.43E-09	2.20E-10	131

Pennes' bioheat equation. Secondly, an application was presented aiming to pair with cubic spline interpolation to allow a user to reconstruct an interface from an image. Next, the cubic spline reconstructions were tested and analyzed; the results determined that our implementation of NCSI does not currently achieve the necessary accuracy to be implemented into the AMIB method. Following, five numerical treatments necessary to implementing an interface into the AMIB method were described in detail. Afterwards, the paper transitioned into describing and demonstrating the AMIB method. A simplified explanation of the numerical method formulation was presented; then the numerical method results were put forth and analyzed.

The results demonstrated that the AMIB method is capable of achieving its goal of solving immersed problems over highly irregular domains while maintaining fourth-order accuracy in space and second-order accuracy in time. Additionally, the AMIB method demonstrated unconditional stability, the ability to handle versatile boundary conditions, and high efficiency largely due to the implementation of the Fast Fourier Transform when solving the augmented system.

Further research is required to improve and expand our solving of the Heat equation across irregularly-shaped domains. First and foremost, the application introduced aiming to allow a user to reconstruct an interface from an image is still very early in development. Further development of the graphical user interface and the implementation of a parametric cubic spline algorithm are necessary modifications to implement the application into the AMIB method. The five numerical treatments are still under development; the efficiency, accuracy, and stability of the numerical treatments need to be further researched. Though the AMIB method has demonstrated success in solving immersed problems over irregular two-dimensional domains, the method still requires significant research and development. Namely, the AMIB method struggles with highly irregular interfaces with thin, sharp corners. Additionally, the AMIB method is not yet developed enough to solve immersed problems on complicated three-dimensional domains. Significant research is to continue in solving the Heat equation across irregularly domains.

Bibliography

- [1] Pennes, H. H. "Analysis of Tissue and Arterial Blood Temperatures in the Resting Human Forearm." *Journal of Applied Physiology* (1985), vol. 85, no. 1, 1998, pp. 5-34, <https://doi.org/10.1152/jappl.1998.85.1.5>.
- [2] Li, Chuan, et al. "A Fast Sine Transform Accelerated High-Order Finite Difference Method for Parabolic Problems over Irregular Domains." *Journal of Scientific Computing*, vol. 95, no. 2, 2023, pp. 49-, <https://doi.org/10.1007/s10915-023-02177-7>.
- [3] Ren, Yiming, et al. "A FFT Accelerated High Order Finite Difference Method for Elliptic Boundary Value Problems over Irregular Domains." *Journal of Computational Physics*, vol. 448, 2022, pp. 110762-, <https://doi.org/10.1016/j.jcp.2021.110762>.